

VLLM专题（二十一）——分布式推理与服务



大模型专题系列 专栏收录该内容

¥49.90
¥99.00

111 篇文章

已订阅

8折续

👑 您已是超级会员，正在免费阅读会员专享内容

[查看更多超级会员权益](#) >

1. 如何决定分布式推理策略？

在深入探讨分布式推理和服务之前，我们首先需要明确何时使用分布式推理以及可用的策略是什么。常见的做法如下：

单 GPU（无需分布式推理）：

如果你的模型可以放入单个 GPU 中，那么你可能不需要使用分布式推理。直接使用单个 GPU 运行推理即可。

单节点多 GPU（张量并行推理）：

如果你的模型太大，无法放入单个 GPU，但可以放入单个节点的多个 GPU 中，你可以使用张量并行（Tensor Parallelism）。张量并行大小是你希望使用的 GPU 数量。例如，如果你的单个节点有 4 个 GPU，可以将张量并行大小设置为 4。

多节点多 GPU（张量并行加流水线并行推理）：

如果你的模型太大，无法放入单个节点，你可以结合使用张量并行和流水线并行（Pipeline Parallelism）。张量并行大小是每个节点中使用的 GPU 数量，流水线并行大小是你希望使用的节点数量。例如，如果你有 2 个节点共 16 个 GPU（每个节点 8 个 GPU），可以将张量并行大小设置为 8，流水线并行大小设置为 2。

简而言之，你应该增加 GPU 和节点的数量，直到有足够的 GPU 内存来容纳模型。张量并行大小应为每个节点中的 GPU 数量，流水线并行大小应为节点数量。

在增加足够的 GPU 和节点以容纳模型后，你可以先运行 vLLM，它会打印一些日志，例如 `# GPU blocks: 790`。将该数字乘以 16（块大小），你可以大致得到当前配置下可以服务的最大 token 数量。如果这个数字不满足需求（例如，你希望更高的吞吐量），可以进一步增加 GPU 或节点的数量，直到块数量足够。

注意

有一个特殊情况：如果模型可以放入单个节点的多个 GPU 中，但 GPU 数量无法均匀分配模型大小，你可以使用流水线并行，它会沿模型层进行分割并支持不均匀的分割。在这种情况下，张量并行大小应为 1，流水线并行大小应为 GPU 数量。

2. 在单个节点上运行 vLLM

vLLM 支持分布式张量并行（Tensor-Parallel）和流水线并行（Pipeline-Parallel）推理与服务。目前，我们支持 Megatron-LM 的张量并行算法。我们使用 **Ray** 或 Python 原生的 **multiprocessing** 来管理分布式运行时环境。在单个节点上部署时可以使用 multiprocessing，而多节点推理目前需要 Ray。

如果未在 Ray 放置组中运行，并且同一节点上有足够的 GPU 用于配置的 `tensor_parallel_size`，则默认使用 multiprocessing，否则将使用 Ray。可以通过 LLM 类的 `distributed_executor_backend` 参数或 API 服务器的 `--distributed-executor-backend` 参数覆盖此默认行为。将其设置为 `mp` 以使用 multiprocessing，或设置为 `ray` 以使用 Ray。在使用 multiprocessing 时，无需安装 Ray。

使用 LLM 类运行多 GPU 推理

设置 `tensor_parallel_size` 参数为你希望使用的 GPU 数量。例如，在 4 个 GPU 上运行推理：

```
from vllm import LLM
llm = LLM("facebook/opt-13b", tensor_parallel_size=4)
output = llm.generate("San Francisco is a")
```

运行多 GPU 服务

在启动服务器时传入 `--tensor-parallel-size` 参数。例如，在 4 个 GPU 上运行 API 服务器：

```
vllm serve facebook/opt-13b \
--tensor-parallel-size 4
```

你还可以额外指定 `--pipeline-parallel-size` 以启用流水线并行。例如，在 8 个 GPU 上使用流水线并行和张量并行运行 API 服务器：

```
vllm serve gpt2 \
--tensor-parallel-size 4 \
--pipeline-parallel-size 2
```

3. 在多个节点上运行 vLLM

如果单个节点没有足够的 GPU 来容纳模型，你可以使用多个节点运行模型。确保所有节点的执行环境相同非常重要，包括模型路径和 Python 环境。推荐的方法是使用 Docker 镜像来确保环境一致，并通过将它们映射到相同的 Docker 配置中来隐藏主机的异构性。

第一步：启动容器并将其组织成集群

我们提供了辅助脚本 `examples/online_serving/run_cluster.sh` 来启动集群。请注意，此脚本启动的 Docker 没有运行性能分析和跟踪工具时访问 GPU 性能计数器所需的权限。为此，可以在 `docker run` 命令中使用 `--cap-add` 选项为 Docker 容器添加 `CAP_SYS_ADMIN` 权限。

选择一个节点作为头节点（head node），并运行以下命令：

```
bash run_cluster.sh \  
  vllm/vllm-openai \  
  ip_of_head_node \  
  --head \  
  /path/to/the/huggingface/home/in/this/node \  
  -e VLLM_HOST_IP=ip_of_this_node
```

在其他工作节点（worker nodes）上，运行以下命令：

```
bash run_cluster.sh \  
  vllm/vllm-openai \  
  ip_of_head_node \  
  --worker \  
  /path/to/the/huggingface/home/in/this/node \  
  -e VLLM_HOST_IP=ip_of_this_node
```

然后你将获得一个由容器组成的 Ray 集群。请注意，你需要保持运行这些命令的 shell 处于活动状态以维持集群。任何 shell 断开连接都会终止集群。此外，请注意参数 `ip_of_head_node` 应为头节点的 IP 地址，且所有工作节点都可以访问该地址。每个工作节点的 IP 地址应在 `VLLM_HOST_IP` 环境变量中指定，并且每个工作节点的 IP 地址应不同。请检查集群的网络配置，以确保节点可以通过指定的 IP 地址相互通信。

警告

由于这是一个由容器组成的 Ray 集群，所有后续命令都应在容器内执行，否则你将在主机上执行命令，而主机并未连接到 Ray 集群。要进入容器，可以使用以下命令：

```
docker exec -it <container_name_or_id> /bin/bash
```

检查 Ray 集群状态

在任何节点上，使用以下命令进入容器：

```
docker exec -it <container_name_or_id> /bin/bash
```

然后执行以下命令检查 Ray 集群的状态：

```
ray status  
ray list nodes
```

你应该看到正确数量的节点和 GPU。

在容器内运行 vLLM

再次进入容器后，你可以像在单个节点上一样使用 vLLM。常见的做法是将张量并行大小设置为每个节点中的 GPU 数量，流水线并行大小设置为节点数量。例如，如果你有 2 个节点共 16 个 GPU（每个节点 8 个 GPU），可以设置如下：

```
vllm serve /path/to/the/model/in/the/container \  
--tensor-parallel-size 8 \  
--pipeline-parallel-size 2
```

你也可以仅使用张量并行而不使用流水线并行，只需将张量并行大小设置为集群中的 GPU 总数。例如，如果你有 2 个节点共 16 个 GPU，可以设置如下：

```
vllm serve /path/to/the/model/in/the/container \  
--tensor-parallel-size 16
```

优化跨节点通信

为了使张量并行高效运行，应确保节点之间的通信高效，例如使用高速网络卡（如 Infiniband）。要正确设置集群以使用 Infiniband，请在 `run_cluster.sh` 脚本中附加额外的参数，例如 `--privileged -e NCCL_IB_HCA=mlx5`。请联系系统管理员以获取有关如何设置这些标志的更多信息。

确认 Infiniband 是否正常工作的一种方法是设置 `NCCL_DEBUG=TRACE` 环境变量运行 vLLM，例如：

```
NCCL_DEBUG=TRACE vllm serve ...
```

然后检查日志中的 NCCL 版本和使用的网络。如果日志中出现 `[send] via NET/Socket`，则表示 NCCL 使用原始 TCP Socket，这对于跨节点张量并行效率较低。如果日志中出现 `[send] via NET/IB/GDRDMA`，则表示 NCCL 使用 Infiniband 和 GPU-Direct RDMA，这是高效的。

警告

启动 Ray 集群后，最好检查节点间的 GPU-GPU 通信。设置通信可能比较复杂，请参考 **sanity check 脚本** 获取更多信息。如果需要为通信配置设置一些环境变量，可以将它们附加到 `run_cluster.sh` 脚本中，例如 `-e NCCL_SOCKET_IFNAME=eth0`。请注意，在 shell 中设置环境变量（例如 `NCCL_SOCKET_IFNAME=eth0 vllm serve ...`）仅对同一节点中的进程有效，而对其他节点中的进程无效。建议在创建集群时设置环境变量。更多信息请参阅 [Issue #6803](#)。

警告

1. 确保模型已下载到所有节点
2. 请确保模型已下载到所有节点（路径相同），或者模型已下载到所有节点均可访问的分布式文件系统中。
3. 使用 HuggingFace 仓库 ID 时附加 Token 当你使用 HuggingFace 仓库 ID 引用模型时，应将 HuggingFace token 附加到 `run_cluster.sh` 脚本中，例如 `-e HF_TOKEN=<your_token>`。推荐的方法是先下载模型，然后使用路径引用模型。

警告

如果你不断收到错误消息 `Error: No available node types can fulfill resource request`，但集群中有足够的 GPU，可能是因为你的节点有多个 IP 地址，而 vLLM 无法找到正确的 IP 地址，尤其是在使用多节点推理时。请确保 vLLM 和 Ray 使用相同的 IP 地址。你可以在 `run_cluster.sh` 脚本中设置 `VLLM_HOST_IP` 环境变量为正确的 IP 地址（每个节点不同！），并通过 `ray status` 和 `ray list nodes` 检查 Ray 使用的 IP 地址。更多信息请参阅 [Issue #7815](#)。